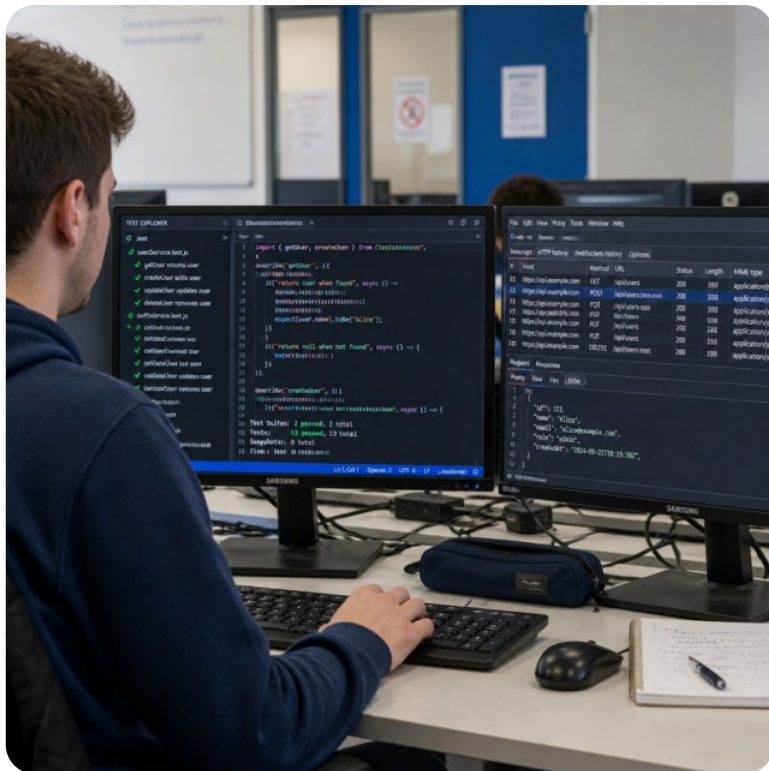


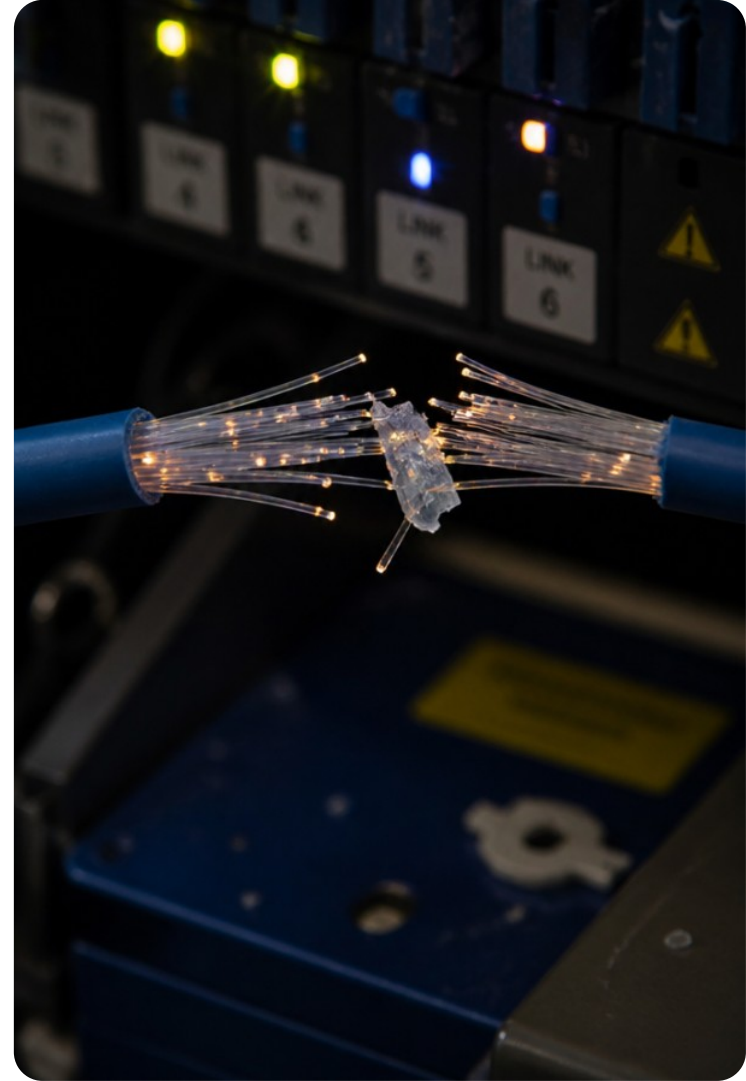


Testes de Integração Avançados e Mock de APIs

Domine MSW, estado compartilhado e assincronismo no frontend profissional.

O Desafio da Rede Real

O que acontece quando sua suite de testes depende de um servidor que caiu?



Objetivos da Aula de Hoje

Resultados Esperados

Ao final deste bloco, você terá dominado habilidades essenciais de integração:

Interceptar requisições HTTP com **Mock Service Worker**.

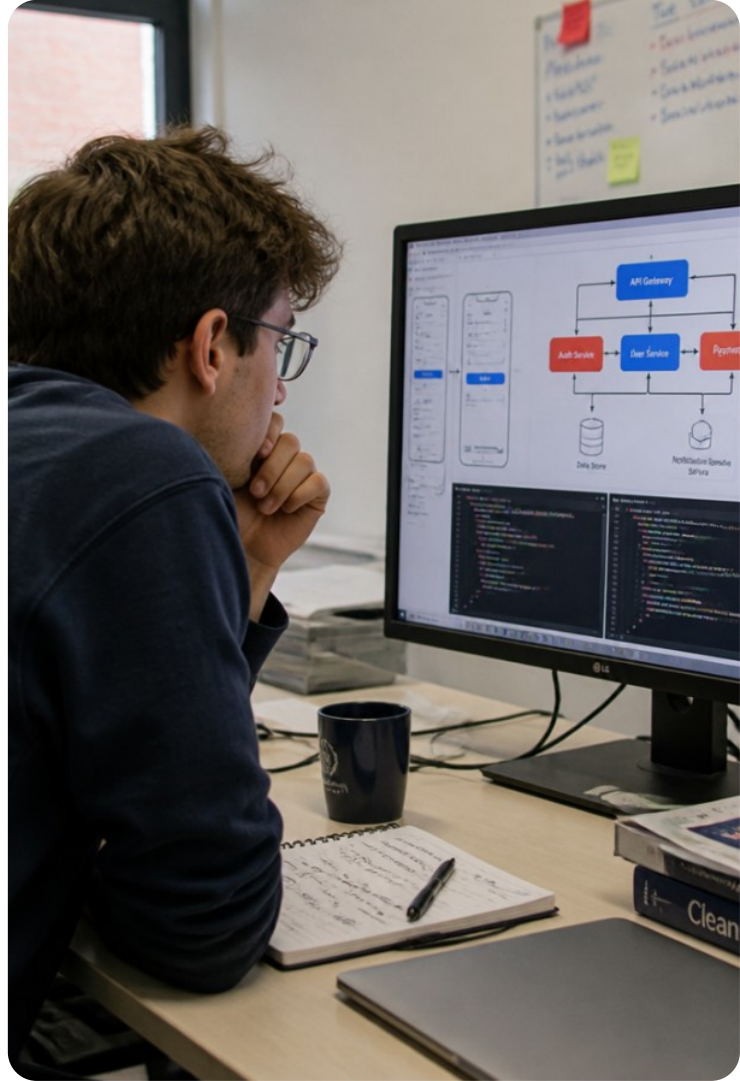
Validar **estados compartilhados** em Context API e storage.

Controlar fluxos assíncronos e evitar **race conditions**.



Lembre-se

Testes de integração garantem que peças isoladas funcionem perfeitamente juntas.



Vocabulário Essencial de Integração



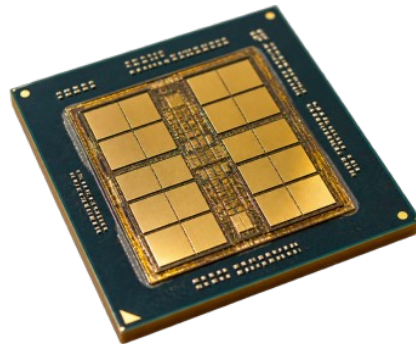
Interceptação

Captura de chamadas HTTP na camada de rede antes de saírem do ambiente.



Assincronismo

Operações independentes do tempo linear que dependem de promessas e eventos.



Persistência

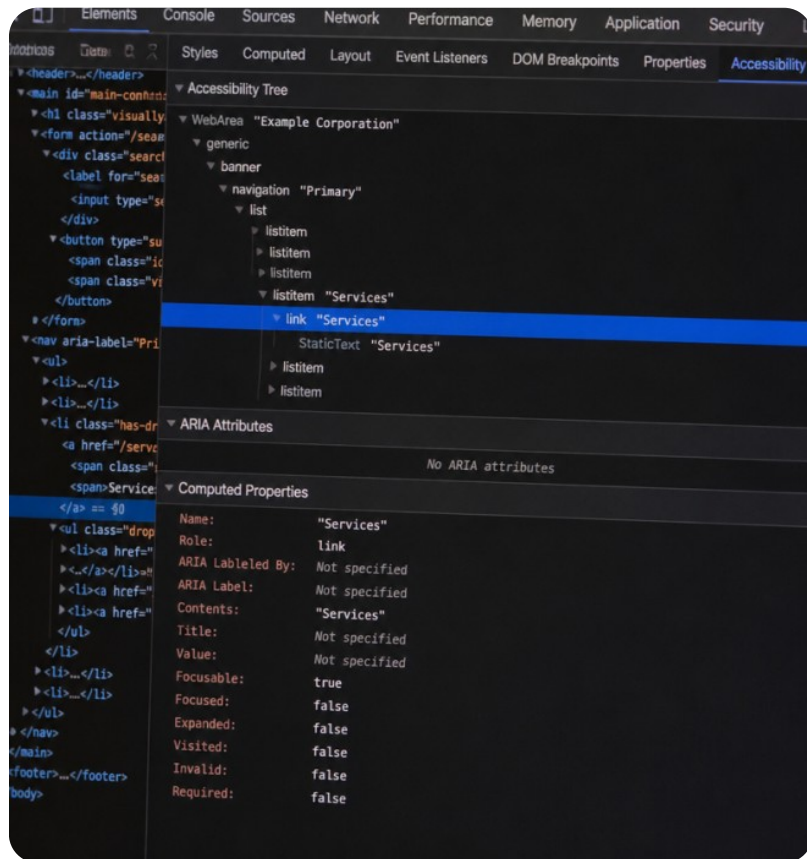
Manutenção de dados intactos após recarregamentos, usando APIs como



Estado

Memória dinâmica e dados reativos mantidos por componentes ativos.

Recapitulação: Testing Library



REVISÃO

Consultas no DOM

Priorizamos acessibilidade e interação real:

- busca síncrona; erro se ausente.
- retorna `null` se ausente; ideal para remoções.
- retorna `Promise`; aguarda elemento surgir.



Ponto-chave

Simulamos eventos via `fireEvent`.

Quiz: Seleção de Queries DOM



Pergunta 1:

Qual query deve ser usada para verificar que um modal de erro desapareceu da tela?

Pergunta 2:

Quando é obrigatório recorrer a uma query do tipo findBy?

Pergunta 3:

Por que getByRole é preferível em relação a getId?

Quiz: Seleção de Queries DOM



Resposta 1:

queryBy (ou queryByRole), pois retorna null quando o elemento não existe, sem quebrar o teste com exceção.

Resposta 2:

Quando o elemento aparece de maneira assíncrona após uma requisição ou atualização de estado.

Resposta 3:

Porque valida a acessibilidade semântica real experimentada pelo usuário, em vez de depender de detalhes de implementação.

O Limite dos Componentes Isolados

INTEGRAÇÃO

A Complexidade Real

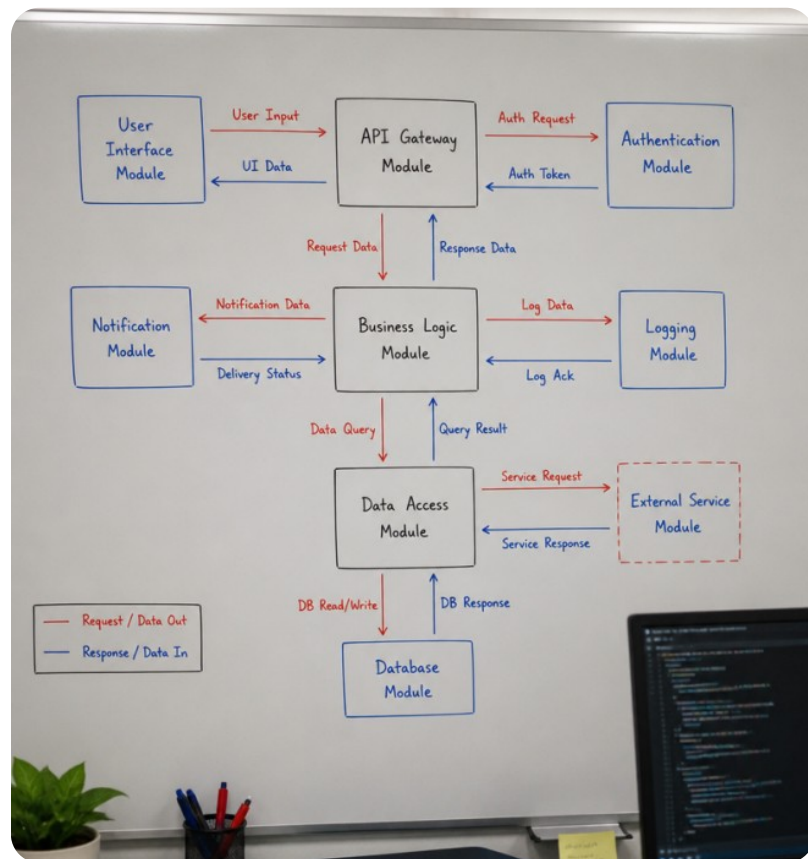
Testar componentes isolados é simples. Em apps reais, eles operam em conjunto:

- Filhos alteram estados globais.
- Contextos de auth definem rotas.
- Requisições HTTP sincronizam dados.

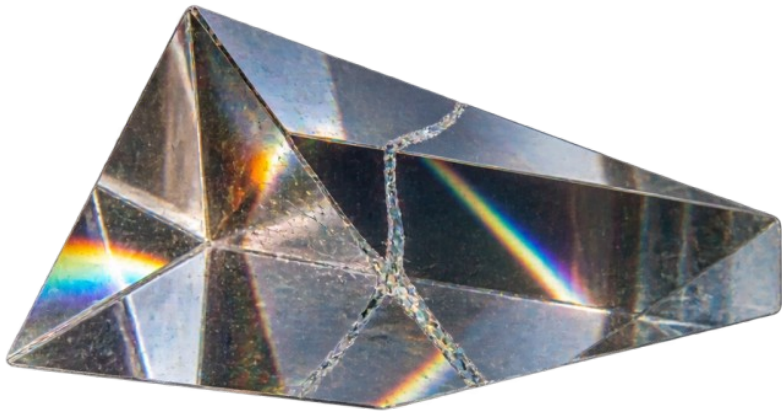


Atenção

Testes unitários ignoram falhas em contratos de API.



O Dilema dos Mocks Tradicionais



Mockar fetch / axios

Substituir acopla o teste à biblioteca usada e mascara erros reais de parsing, headers e URLs mal formatadas.



Interceptação de Rede

A aplicação executa seu código HTTP real sem alterações, enquanto o interceptor atua no nível de rede simulado.

Apresentando o MSW

FERRAMENTA

Mock Service Worker

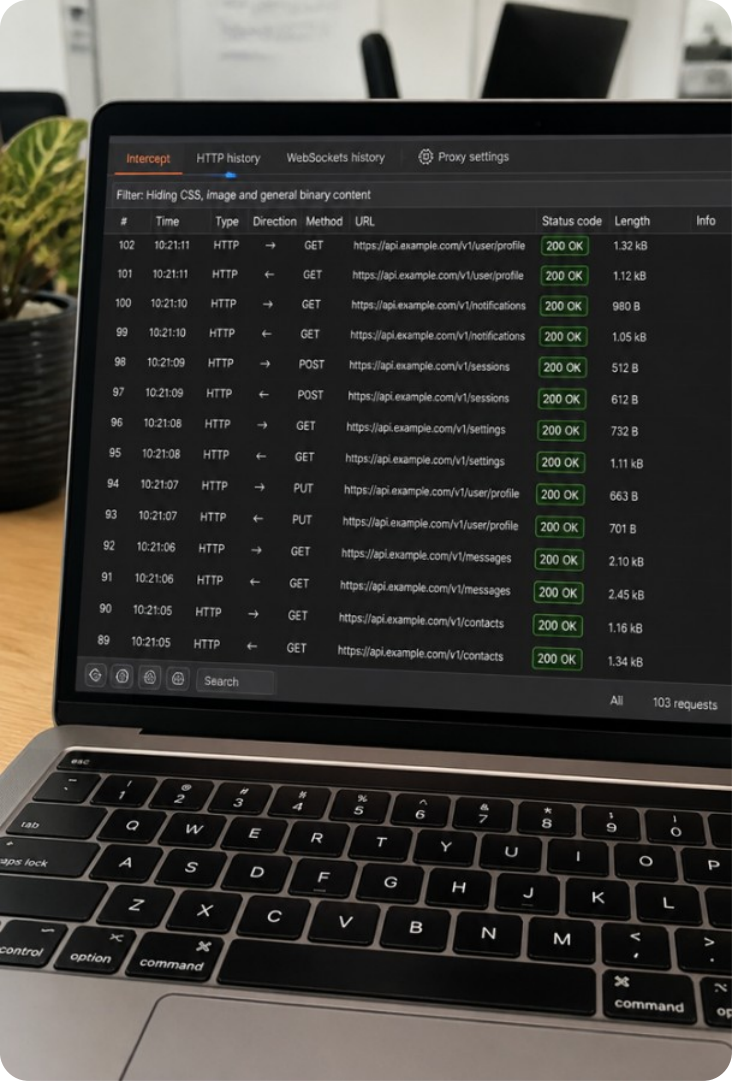
O **MSW** intercepta requisições via Service Workers (browser) ou interceptors (Node.js):

- Mantém o código-fonte original.
- Intercepta REST e GraphQL nativos.
- Gerencia headers, status e payloads reais.



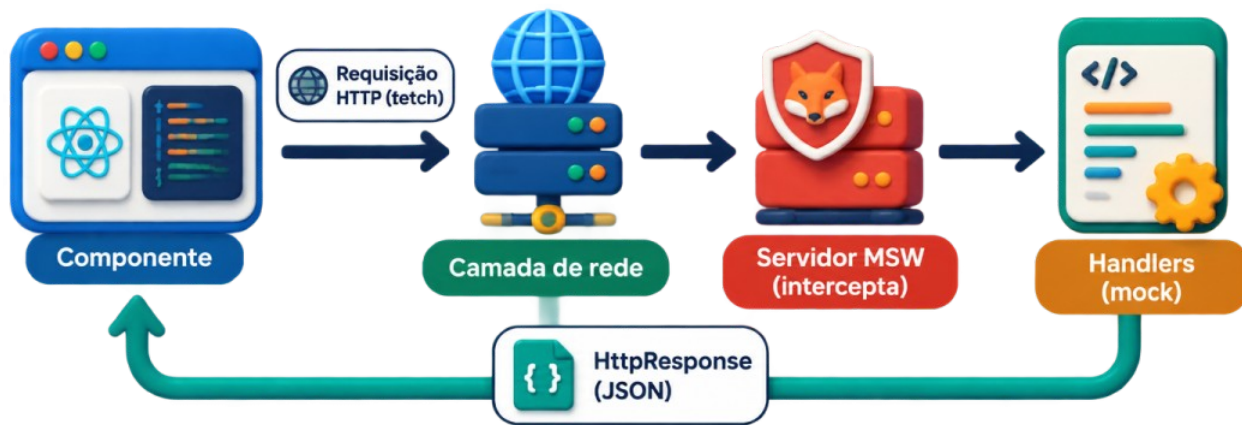
Ponto-chave

Componentes usam o cliente HTTP de produção, garantindo fidelidade total.



Arquitetura do MSW nos Testes

O fluxo de execução do MSW intercepta chamadas na camada de rede em tempo de teste:



O componente nunca sabe que a resposta veio de um mock local em vez do servidor.

Configurando Handlers com MSW

CÓDIGO

Declarando Manipuladores

Definimos manipuladores utilizando a API declarativa e :

Mapeamos endpoints específicos ().

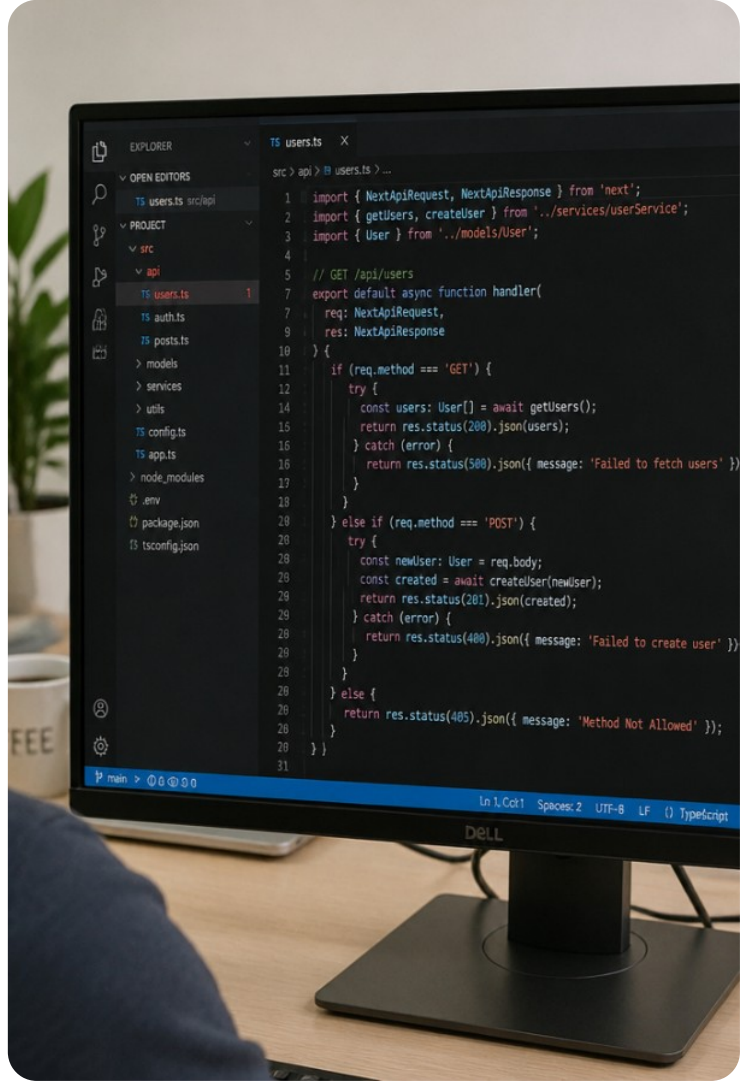
Retornamos respostas formatadas com .

Configuramos o servidor de teste com .



Exemplo

```
http.get('/api/produtos', () =>
  HttpResponse.json([
    { id: 1, nome: 'Teclado' }
  ]))
```



Ciclo de Vida do Servidor MSW



server.listen()

Inicia a escuta antes de todos os testes no hook .

server.resetHandlers()

Restaura os mocks originais no evitando contaminação.

server.close()

Encerra o servidor de interceptação no hook da suite.

Checagem: Vantagens do MSW

Qual é a principal vantagem técnica de adotar o MSW em relação a espionar e mockar o axios diretamente?

1. Garantir que os testes executem mais rápido que testes unitários puros
2. Impedir que componentes React executem ciclos de renderização
3. Exigir um servidor backend em produção em tempo de teste
4. Interceptar chamadas na camada de rede sem alterar a implementação do cliente HTTP

Checagem: Vantagens do MSW



Qual é a principal vantagem técnica de adotar o MSW em relação a espionar e mockar o axios diretamente?

1. Garantir que os testes executem mais rápido que testes unitários puros
2. Impedir que componentes React executem ciclos de renderização
3. Exigir um servidor backend em produção em tempo de teste
4. Interceptar chamadas na camada de rede sem alterar a implementação do cliente HTTP

Simulando Respostas Positivas

CENÁRIOS

O Caminho Feliz (Status 200)

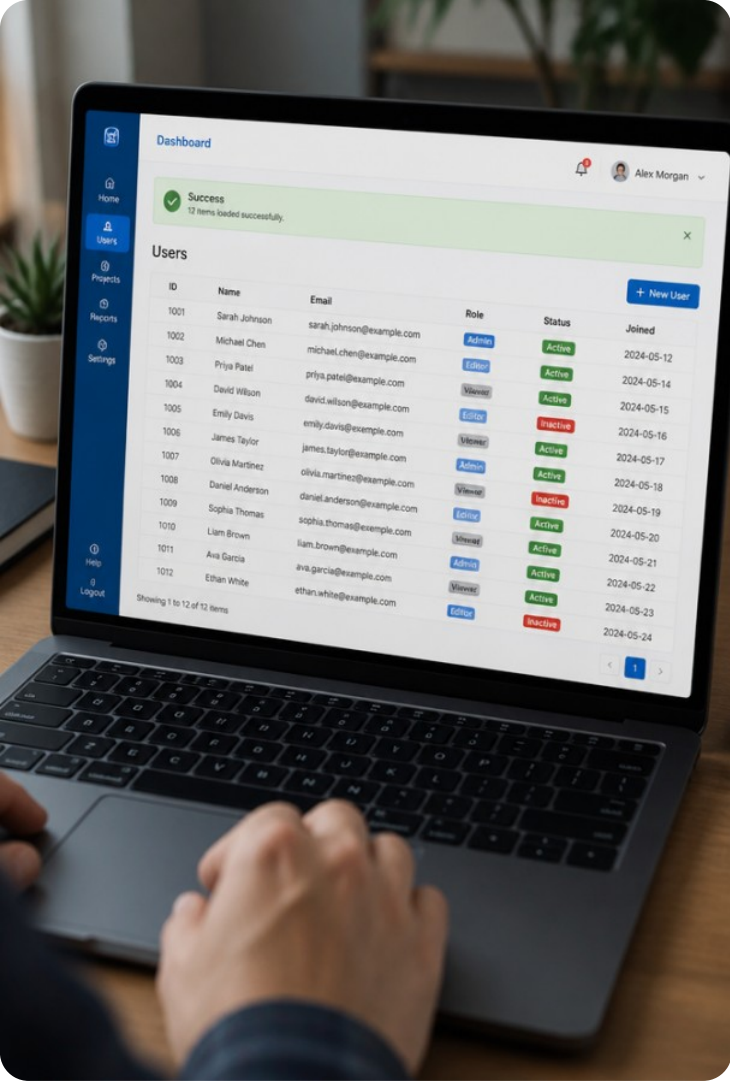
Testar o sucesso valida a renderização de tabelas:

- O componente exibe estado inicial vazio.
- Dispara o request ao carregar.
- Preenche o DOM com os dados do mock.

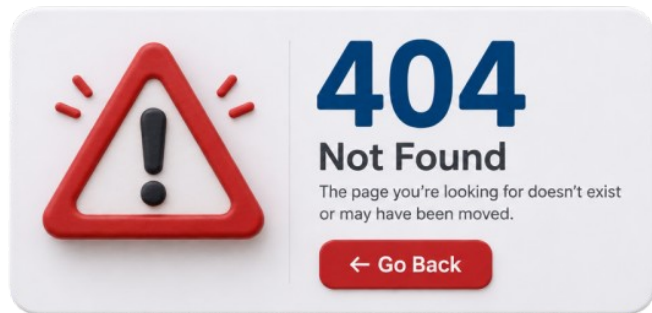


Ponto-chave

Use `findByRole` para esperar a lista ser populada após o request.

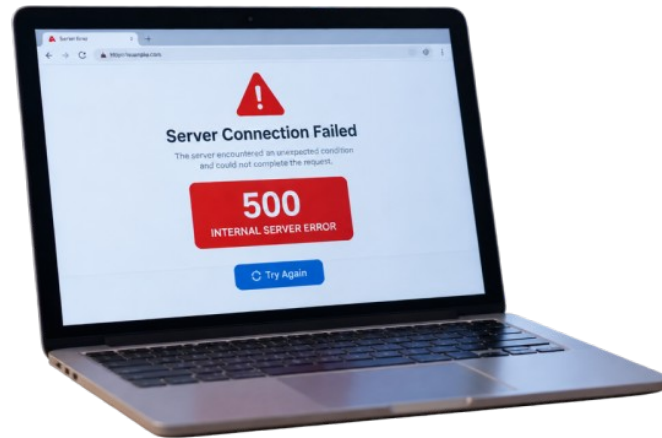


Tratando Falhas: 404 e 500



Recurso Inexistente (404)

Sobrescreva o handler com para validar mensagens amigáveis de dados não localizados.



Falha Interna (500)

Simule quedas do servidor para garantir que botões de 'Tentar Novamente' e alertas de erro funcionem.

Simulação de Latência de Rede

CENÁRIOS

Atrasos Realistas

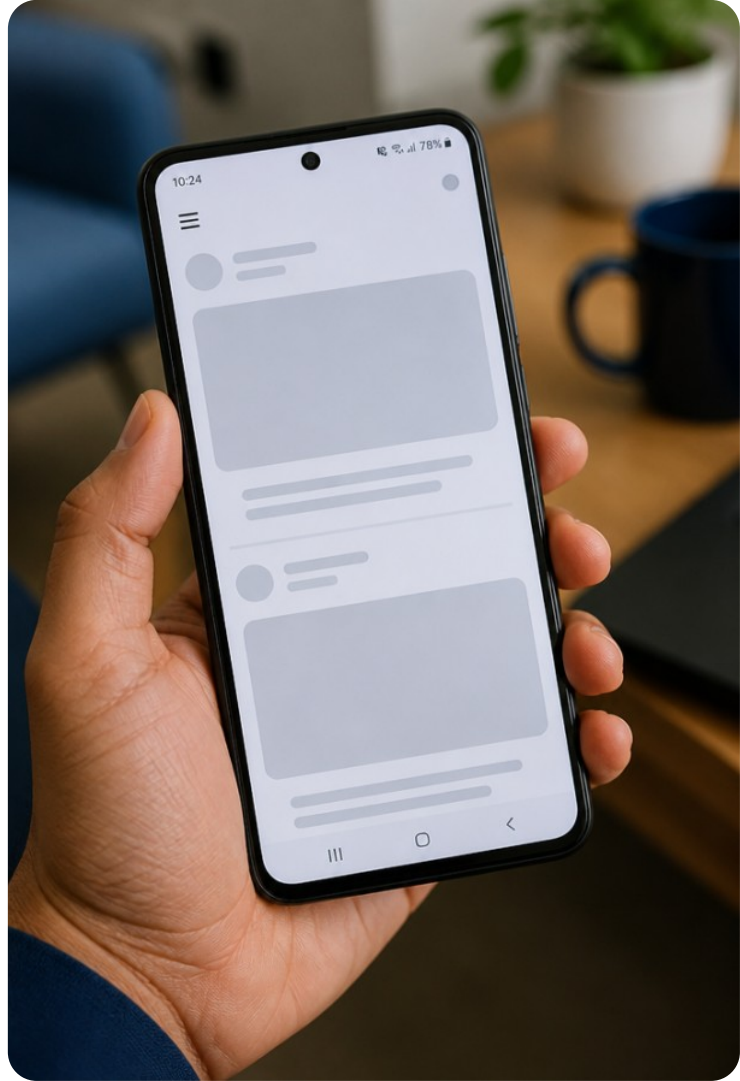
Use no MSW para testar estados:

- Botões desativados (evita duplo clique).
- Skeletons de carregamento.
- Feedbacks visuais em esperas longas.



Exemplo

```
await delay(150); return  
HttpResponse.json({ status: 'ok' });
```



Gerenciamento de Loaders e Spinners

Para testar ciclos visuais completos com spinners assíncronos, siga três etapas ordenadas:

A diagram showing three sequential steps in a process flow. Each step is represented by a red-outlined chevron pointing to the right, connected by a continuous line. The steps are labeled 1. Início, 2. Transição, and 3. Conclusão.

1. Início

Dispare o evento de submissão e confirme o spinner com .

2. Transição

Use para aguardar o desaparecimento do spinner.

3. Conclusão

Valide os dados finais na tela com .

Análise: Asserções de Desaparecimento

Devemos usar `getByRole('progressbar')` imediatamente após a requisição para verificar que o spinner sumiu da tela.



VERDADE
IRO



FALSO



Prepare-se para explicar o seu raciocínio.

Análise: Asserções de Desaparecimento



Devemos usar `getByRole('progressbar')` imediatamente após a requisição para verificar que o spinner sumiu da tela.

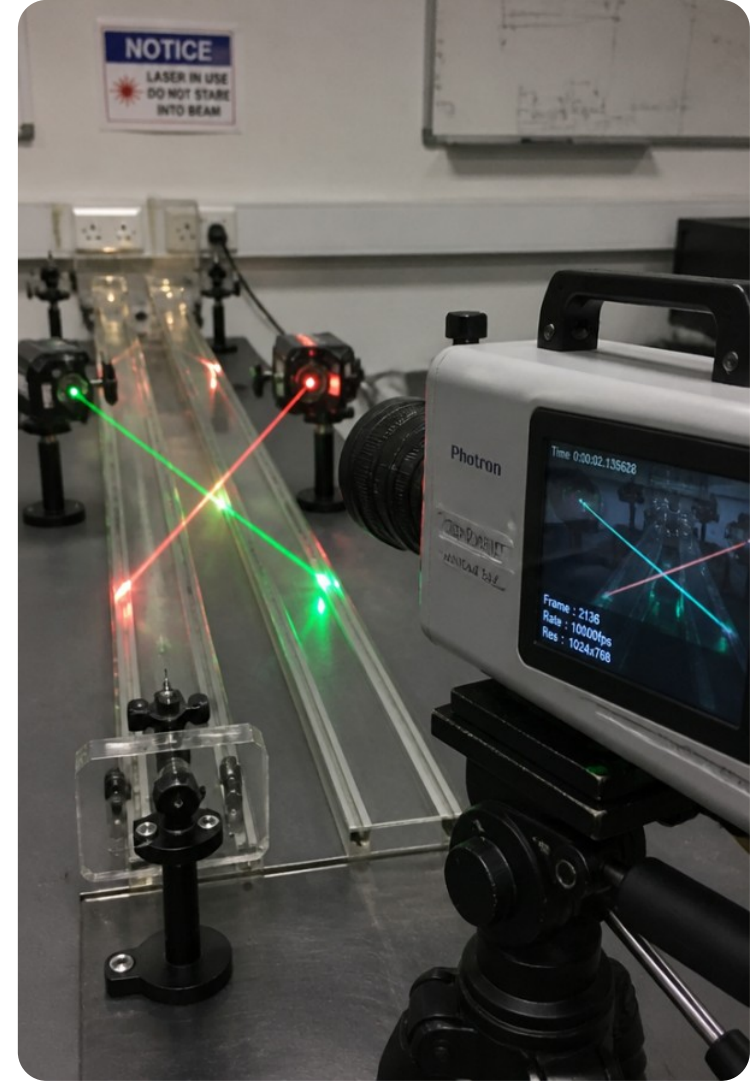


Por que é isso?

`getByRole` lança uma exceção se o elemento já desapareceu; deve-se usar `queryByRole` ou `waitForElementToBeRemoved`.

O Perigo do Tempo Desordenado

E se a primeira requisição demorar mais para responder do que a segunda?



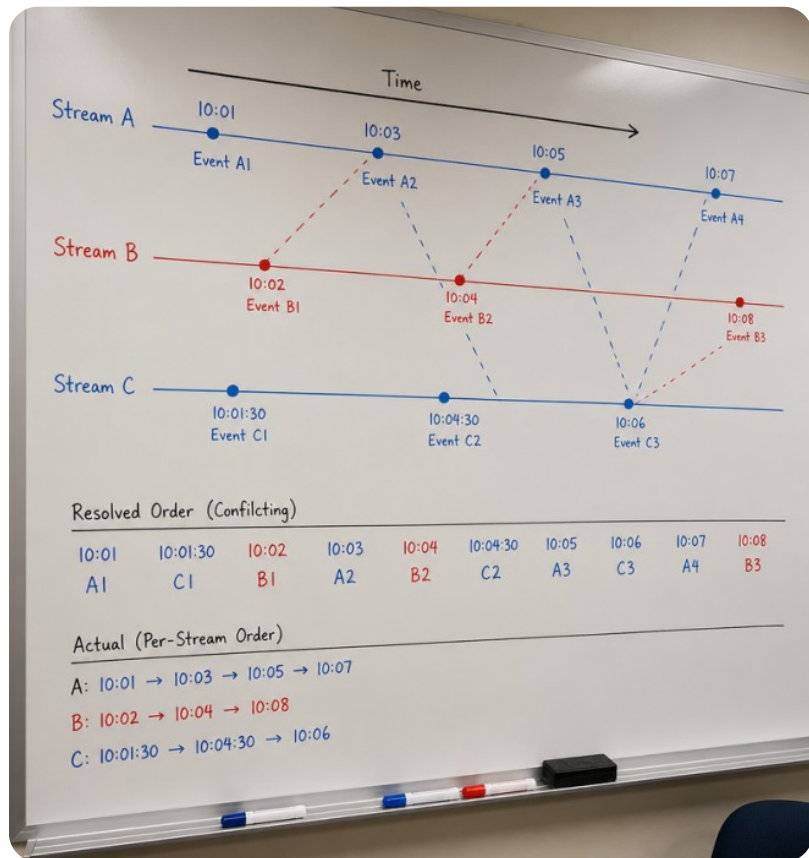
Prevenção de Race Conditions

ASSINCRONISMO

Respostas Fora de Ordem

Race conditions ocorrem quando requisições finalizam fora de ordem:

- Busca rápida gera chamadas múltiplas.
- Resposta antiga sobrescreve a recente.
- Interface exibe dados obsoletos.



Atenção

Simule latências e valide cancelamento via `AbortController`.

Vazamento de Memória em Testes

ASSINCRONISMO

Aviso 'Not-Wrapped in act'

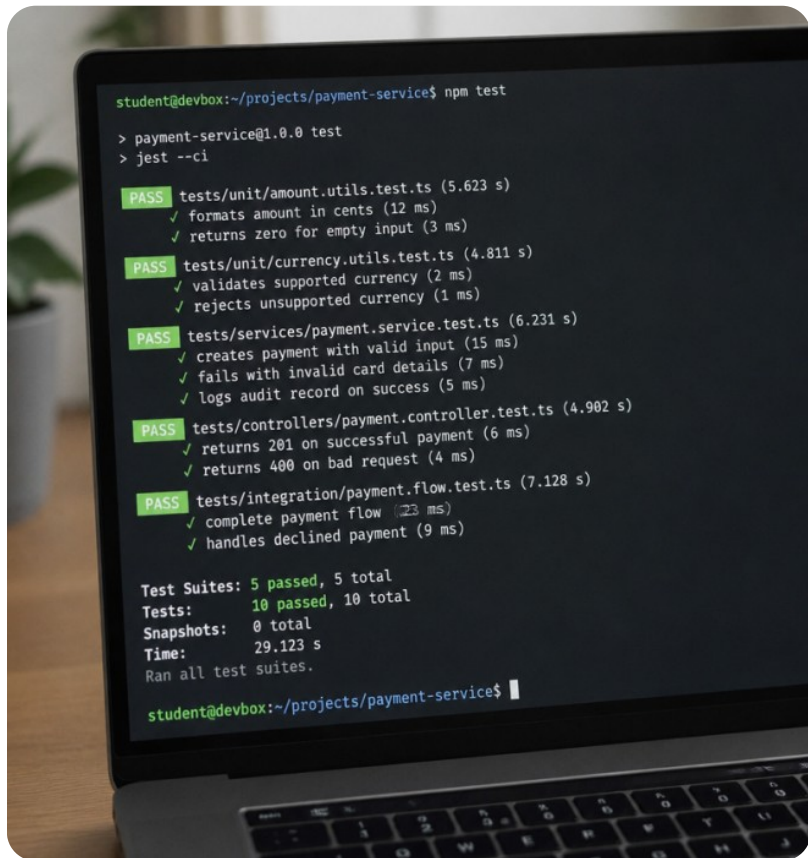
Ocorre ao desmontar componentes com promessas pendentes:

- Chamadas HTTP resolvem após o teste.
- tenta rodar em nó destruído.
- Gera instabilidade e testes intermitentes.



Ponto-chave

Aguarde sempre a conclusão de promessas antes de finalizar o teste.



Demonstração de Integração com MSW



Arquitetura de interceptação de rede e validação assíncrona com Testing Library:

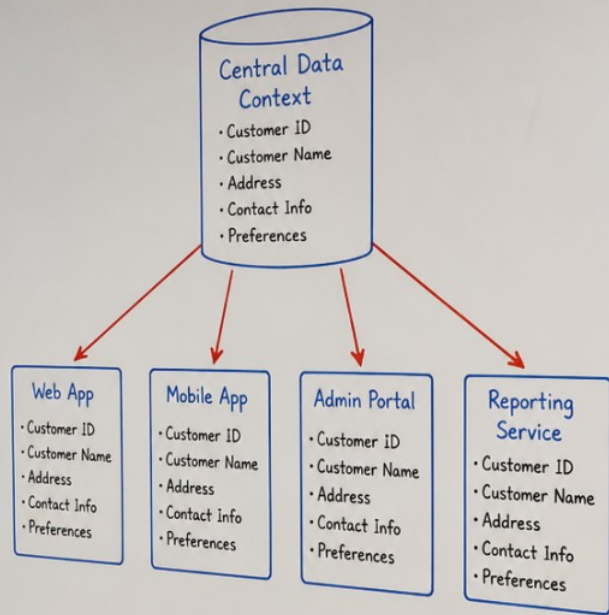
Estado Compartilhado: Context e Redux

ESTADO

Componentes Não Vivem Isolados

Componentes dependem de Provedores globais:

- Dados do usuário (AuthContext).
- Itens do carrinho (CartContext).
- Tema visual e idioma.



Ponto-chave

Use `renderWithProviders()` para encapsular árvores de componentes em testes.

Padrão Custom Render

A melhor prática da Testing Library consiste em encapsular providers em um helper reutilizável:

Estrutura do Helper



Ponto-chave

Substitui repetições desnecessárias de Provedores em centenas de arquivos de teste.

Permite injetar estados iniciais flexíveis para testar perfis de administradores, clientes anônimos ou usuários bloqueados.

Persistência com LocalStorage

PERSISTÊNCIA

Validação de Sessão e Cache

Apps salvam tokens e carrinhos no Web Storage:

No Vitest, mocke .

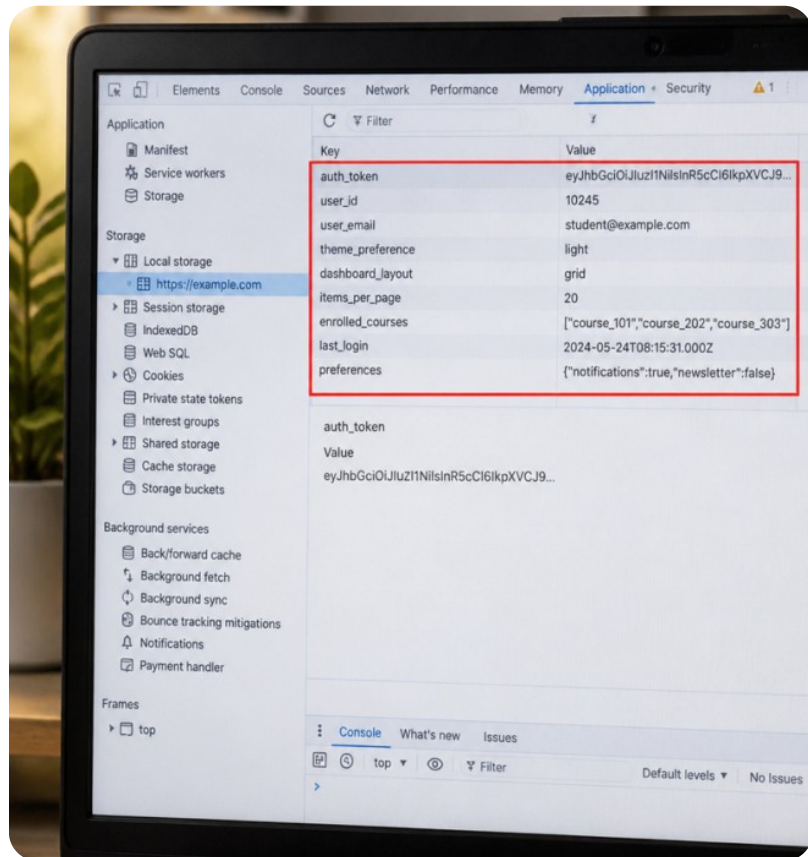
Verifique se salvou dados.

- Valide a hidratação ao carregar storage.



Lembre-se

Use `localStorage.clear()` no `afterEach` para isolar testes.



Boas Práticas de Estado Global

Quais afirmativas sobre testes com estado global e persistência são verdadeiras?
(Mais de uma está correta)

1. Testes de componentes com Context devem sempre acessar diretamente as variáveis internas de state.
2. O localStorage deve ser limpo entre testes para evitar que dados vazem para o próximo caso.
3. É recomendável criar um custom render que envolva componentes com os Providers necessários.
4. O MSW é responsável por salvar e restaurar dados diretamente no localStorage.

Boas Práticas de Estado Global



Quais afirmativas sobre testes com estado global e persistência são verdadeiras?
(Mais de uma está correta)

1. Testes de componentes com Context devem sempre acessar diretamente as variáveis internas de state.
2. O localStorage deve ser limpo entre testes para evitar que dados vazem para o próximo caso.
3. É recomendável criar um custom render que envolva componentes com os Providers necessários.
4. O MSW é responsável por salvar e restaurar dados diretamente no localStorage.

Prática 1: Fluxo de Autenticação

ATIVIDADE PRÁTICA

Roteiro do Teste de Login com MSW

Implementaremos um teste de integração de ponta a ponta para validar a autenticação completa:



1. Setup

Configure o endpoint POST no MSW retornando status 200 e token JWT.

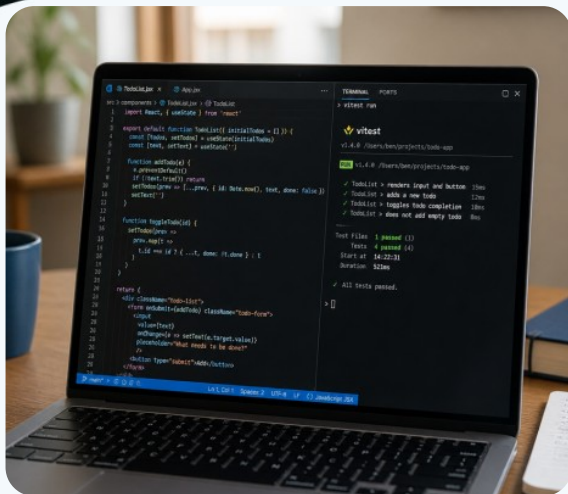
2. Interação

Preencha os campos de email e senha usando e clique no botão Entrar.

3. Asserção

Aguarde com a saudação do usuário no cabeçalho e confirme o token salvo.

Execução da Prática 1: Autenticação



No terminal do VS Code:

1. Em `src/mocks/handlers.js`, crie `http.post('/api/login')` validando `aluno@senai.br` e `123456`.

1. Crie `LoginFlow.test.jsx` e renderize `<App />` via `renderWithProviders()`.

1. Simule digitação e envio do formulário com `user-event`.

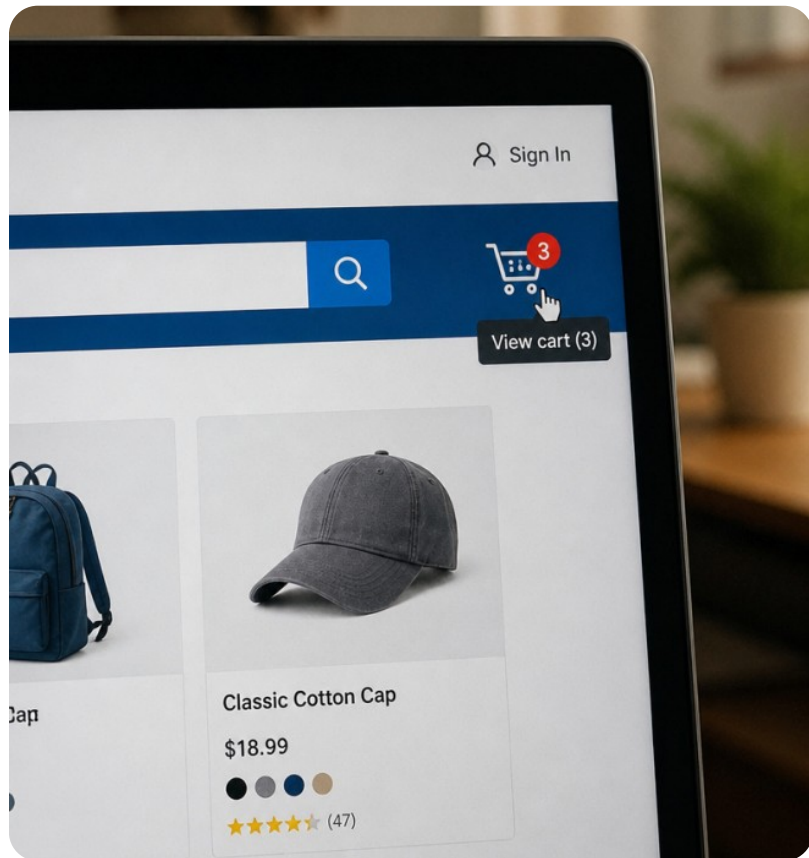
1. Asserções:

- Verifique rota ou 'Bem-vindo, Aluno' via `findByRole`.

- Confirme `localStorage`:
`expect(localStorage.getItem('token')).toBe('mock-jwt-xyz')`.

1. Execute `npx vitest run LoginFlow` e valide sem warnings.

Prática 2: Persistência no Storage



ATIVIDADE PRÁTICA

Cenário: Carrinho Offline

Teste persistência sem conexão:

- Clique em 'Adicionar'.
- Badge deve atualizar para 1.

Salve via JSON no .

- Recarregue: recupere dados salvos.



Exemplo

```
JSON.parse(localStorage.getItem('@carrinho'))
```

Ordem do Teste de Persistência

Ordene as etapas necessárias para testar a recuperação e hidratação do carrinho a partir do localStorage:

Validar com `findByText` que os itens pré-gravados aparecem renderizados na tela

Renderizar o componente do carrinho usando o helper de teste

Gravar dados pré-existent no localStorage com `setItem` antes da montagem

Limpar o storage no hook `afterEach` para isolar a próxima execução

Ordem do Teste de Persistência



Ordene as etapas necessárias para testar a recuperação e hidratação do carrinho a partir do localStorage:

- 1 **Gravar dados pré-existent no localStorage com setItem antes da montagem**
- 2 **Renderizar o componente do carrinho usando o helper de teste**
- 3 **Validar com findByText que os itens pré-gravados aparecem renderizados na tela**
- 4 **Limpar o storage no hook afterEach para isolar a próxima execução**

Manutenibilidade e Performance

ENGENHARIA

Testes Sustentáveis

Evite lentidão em suites de integração:

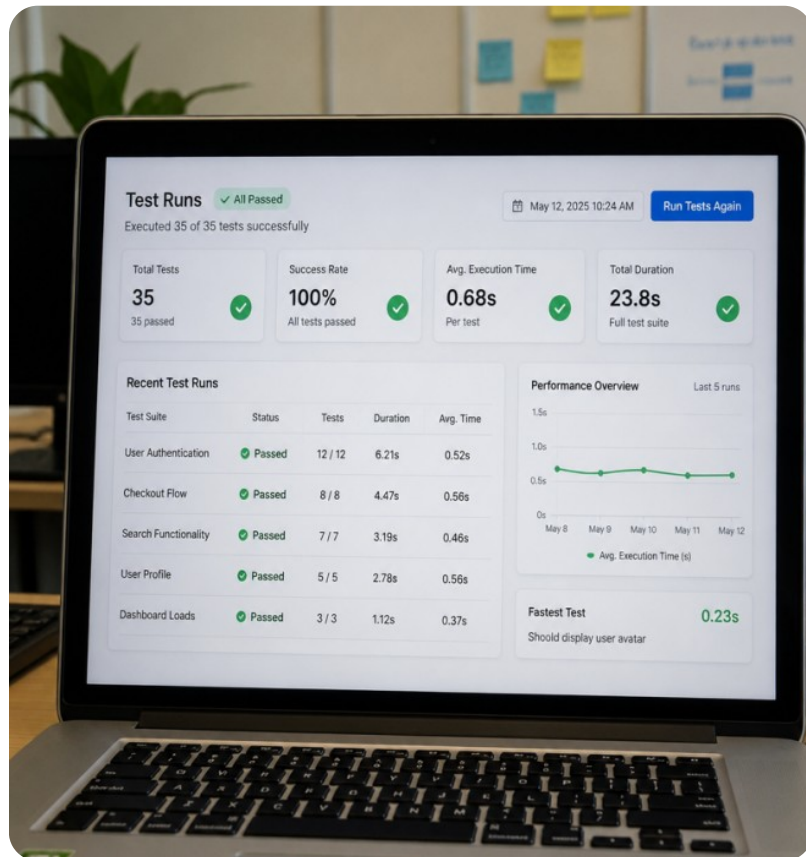
Troque por orientado a eventos.

- Reutilize handlers MSW globais, sobrescrevendo apenas exceções.
- Use paralelismo do Vitest para acelerar a execução.



Ponto-chave

Testes abaixo de 500ms garantem feedback contínuo no Git.



Impacto na Indústria: O Caso Shopify



O Incidente

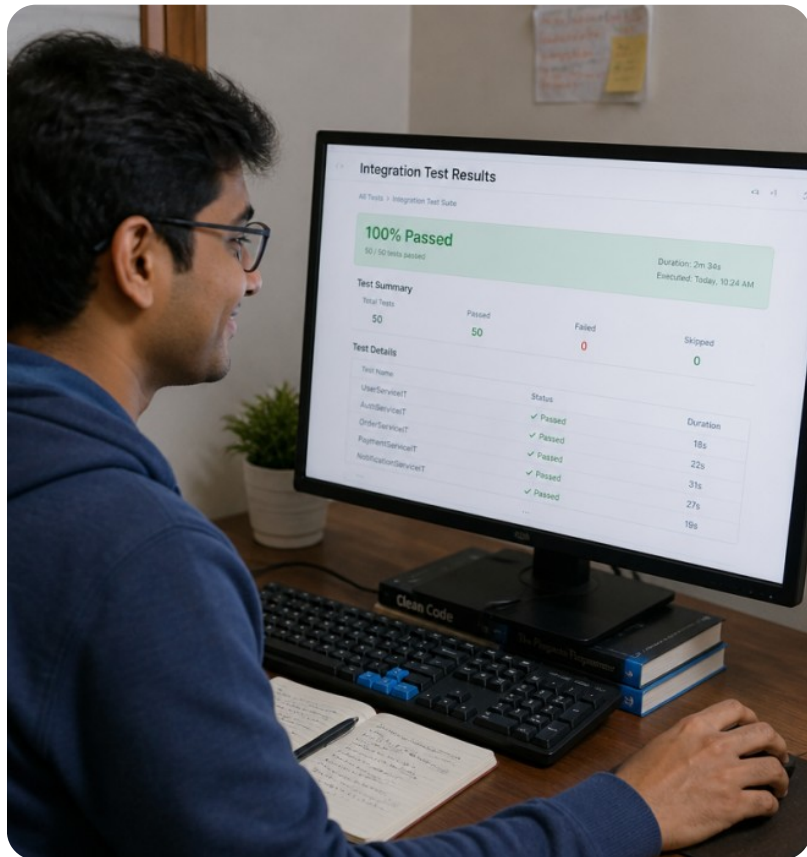
Durante a Black Friday, uma race condition no carrinho gerou 2 horas de indisponibilidade e prejuízo de US\$ 500 mil em vendas.



A Solução

Criação de testes de integração simulando concorrência de rede e latências variadas. Resultado: zero ocorrências similares há 5 anos.

Síntese dos Aprendizados



REVISÃO

Pontos Fundamentais

Dominamos alicerces de integração frontend:

MSW: Intercepta rede com realismo.

- Mocks de erro/latência blindam a UI.

Render helpers: facilitam testes com contextos e storage.



Lembre-se

Testes confiáveis são rápidos, determinísticos e focados no usuário.

Debate: Mock de Rede vs Servidor Real



Em quais momentos da esteira de desenvolvimento uma equipe deve preferir mocks com MSW e quando deve investir em testes contra um servidor backend real de homologação?

Debate: Mock de Rede vs Servidor Real



Você poderia ter dito...

Testes com MSW são ideais no CI e desenvolvimento local por serem rápidos, determinísticos e gratuitos.

Testes E2E são essenciais pré-deploy para auditar migrações de banco e contratos de rede.

Próxima Etapa: Automação E2E

PRÓXIMOS PASSOS

Rumo ao Bloco 06

Com testes unitários e de integração prontos, subimos a pirâmide:

- Cypress: navegação real.
- Fluxos de checkout e pagamentos.
- Depuração com time-travel.



Ponto-chave

No Bloco 06, simularemos cliques e rotas em browsers de produção!

